# Kmeans Text Clustering

## 1. Import Required Libraries

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np
```

### Purpose

### TfidfVectorizer

Converts text into TF-IDF feature vectors.

### cosine_similarity

Measures similarity between two vectors.

### NumPy

Used internally for mathematical operations.

## 2. Training Documents

```
documents = [
    "machine learning and artificial intelligence",
    "deep learning neural networks",
    "natural language processing",
    "football world cup tournament",
    "cricket match highlights",
    "soccer championship players"
]
```

These are the training examples.

### Technology Documents

```
machine learning and artificial intelligence
deep learning neural networks
natural language processing
```

### Sports Documents

```
football world cup tournament
cricket match highlights
soccer championship players
```

---

## 3. Category Labels

```
labels = [
    "Technology",
    "Technology",
    "Technology",
    "Sports",
    "Sports",
    "Sports"
]
```

Each document is assigned its correct category.

| Document | Category |
|---|---|
| **machine learning and artificial intelligence** | Technology |
| **deep learning neural networks** | Technology |
| **natural language processing** | Technology |
| **football world cup tournament** | Sports |
| **cricket match highlights** | Sports |
| **soccer championship players** | Sports |

---

## 4. Convert Text into TF-IDF Vectors

```
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)
```

### Purpose

Machine learning algorithms operate on numbers, not text.

The documents are transformed into TF-IDF vectors.

Example:

```
machine learning and artificial intelligence
```

↓

```
[0.34, 0.51, 0.00, 0.27, ...]
```

Each value indicates the importance of a word.

## 5. What is a Centroid?

A **centroid** is the average vector of all documents belonging to a category.

Example:

```
Technology Documents
        ↓
TF-IDF Vectors
        ↓
Average Vector
        ↓
Technology Centroid
```

Similarly:

```
Sports Documents
        ↓
Average Vector
        ↓
Sports Centroid
```

The centroid acts as a representative of the entire category.

## 6. Get Unique Categories

```
categories = list(set(labels))
```

Creates a list of unique classes:

```
["Technology", "Sports"]
```

## 7. Create a Dictionary for Centroids

```
centroids = {}
```

The structure will be:

```
{
   "Technology": centroid_vector,
   "Sports": centroid_vector
```

```
}
```

# 8. Compute Category Centroids

```
for category in categories:
```

Process each category separately.

## Find Documents of That Category

```
indices = [
    i for i in range(len(labels))
    if labels[i] == category
]
```

For Technology:

```
[0, 1, 2]
```

For Sports:

```
[3, 4, 5]
```

## Compute Average TF-IDF Vector

```
centroid = X[indices].mean(axis=0)
```

Formula:

where:

- (V_i) = TF-IDF vector
- (n) = number of documents

The resulting vector becomes the category prototype.

## Store the Centroid

```
centroids[category] = centroid
```

Example:

```
{
    "Technology": [0.12, 0.32, 0.56, ...],
```

```
    "Sports": [0.44, 0.05, 0.01, ...]
}
```

## 9. Test Document

```
test_doc = [
    "neural network and machine learning"
]
```

This unseen document needs classification.

Words such as:

```
neural
network
machine
learning
```

are closely related to Technology.

---

## 10. Convert Test Document

```
test_vector = vectorizer.transform(test_doc)
```

Transforms the test text into a TF-IDF vector using the vocabulary learned from training data.

## 11. Initialize Classification Variables

```
best_category = None
highest_similarity = -1
```

Purpose:

```
best_category
```

stores the most suitable class.

```
highest_similarity
```

stores the highest similarity score encountered so far.

# 12. Compute Similarity with Each Centroid

```
for category in categories:
```

The test document is compared with every category centroid.

---

## Cosine Similarity

```
similarity = cosine_similarity(
    test_vector,
    centroids[category]
)[0][0]
```

Formula:

where:

- A = Test Document Vector
- B = Category Centroid

Range:

```
1   → Exactly Similar
0   → No Similarity
-1  → Opposite
```

For TF-IDF vectors, values usually lie between:

```
0 and 1
```

# 13. Display Similarities

```
print(
    category,
    "Similarity:",
    round(similarity, 4)
)
```

Example output:

```
Technology Similarity: 0.7423
Sports Similarity: 0.0845
```

## 14. Select Best Category

```
if similarity > highest_similarity:
```

Choose the category with the highest similarity.

```
highest_similarity = similarity
best_category = category
```

This is the classification decision.

## 15. Display Final Prediction

```
print("\nPredicted Category:",
      best_category)
```

Expected output:

```
Technology Similarity: 0.7423
Sports Similarity: 0.0845

Predicted Category: Technology
```

because the test document contains technology-related words.

## Visual Understanding

```
                Test Document
          "neural machine learning"
                       |
                       |
        ------------------------------
        |                            |
Technology Centroid          Sports Centroid
     Similarity                   Similarity
        0.74                         0.08
         |                            |
         -----------Choose-----------
                       ↓
                Technology
```

# Centroid Classifier Algorithm

## Training Phase

```
Documents
      ↓
TF-IDF Conversion
      ↓
Group by Category
      ↓
Compute Average Vector
      ↓
Category Centroids
```

## Prediction Phase

```
New Document
      ↓
TF-IDF Vector
      ↓
Compare with All Centroids
      ↓
Compute Cosine Similarities
      ↓
Select Highest Similarity
       ↓
 Predicted Category
```

# Advantages

- ✅ Very simple implementation
- ✅ Fast training
- ✅ Low memory usage
- ✅ Easy to understand
- ✅ Works well on small datasets

## Limitations

❌ Less accurate than SVM on complex datasets

❌ Assumes one centroid represents the entire category

❌ Sensitive to noisy training data

❌ May struggle when classes overlap significantly

## Difference: Centroid vs SVM

| Feature | Centroid Classifier | SVM |
|---|---|---|
| **Classification Method** | Similarity-based | Hyperplane-based |
| **Training Speed** | Very Fast | Moderate |
| **Complexity** | Simple | Advanced |
| **Memory Usage** | Low | Higher |
| **Accuracy** | Good | Usually Better |

## Real-World Applications

- Email Categorization
- Document Organization
- News Classification
- Information Retrieval
- Search Engines
- Topic Identification

## Time Complexity

Let:

- **N** = Number of documents
- **D** = Number of TF-IDF features

- **C** = Number of categories

### Training

```
TF-IDF Generation : O(N × D)
Centroid Calculation : O(N × D)
```

### Prediction

```
Cosine Similarities : O(C × D)
```

Overall, this classifier is computationally efficient and suitable for many basic NLP classification tasks.

## Key Takeaway

This program implements a **Centroid-Based Text Classifier**. It converts documents into **TF-IDF vectors**, calculates an average vector (centroid) for each category, measures **Cosine Similarity** between a new document and each centroid, and predicts the category with the highest similarity score. This makes it one of the simplest yet effective approaches to text classification.